```
str=("Python is easy to learn")
print(str[0:9])
```

```
Python is
```

```
list1=[1,2,3,3,4,5,5,5,4]
list2=[]
for i in list1:
  if i not in list2:
    list2.append(i)
    list1=list2
print(list2)
```

```
[1, 2, 3, 4, 5]
```

```
n = 123
x = (n)[::-1]
print(x)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_6529/790094385.py in <cell line: 0>()
      1 n = 123
----> 2 x = (n)[::-1]
      3 print(x)

TypeError: 'int' object is not subscriptable
```

```
class Student:
  def __init__(self):
    print("____program started___")

  def show(self):
    print("Iam in student class")

obj=Student()
obj.show()
```

```
____program started___
Iam in student class
```

```
class Student:
  def __init__(self):
    self.name="Karunya"
    self.age=19
    self.percentage=88

  def showInfo(self):
    print("Name:",self.name)
    print("Age:",self.age)
    print("Percentage:",self.percentage)

s=Student()
s.showInfo()
```

```
Name: Karunya
Age: 19
Percentage: 88
```

```
class student:
  def __init__(self,name,age,percentage):
    self.name=name
    self.age=age
    self.percentage=percentage

  def shoeInfo(self):
    print("Name:",self.name)
    print("Age:",self.age)
    print("Percentage:",self.percentage)
```

```
s=student("joy",22,55)
s1=student("Aditya",23,53)
s.shoeInfo()
s1.shoeInfo()
print("Name:",s.name)
```

```
Name: joy
Age: 22
Percentage: 55
Name: Aditya
Age: 23
Percentage: 53
Name: joy
```

```
class student:
  def __init__(self,name,age,percentage):
    self.name=name
    self.age=age
    self.percentage=percentage

  def shoeInfo(self):
    print("Name:",self.name)
    print("Age:",self.age)
    print("Percentage:",self.percentage)

name=input("Enter Name:")
age=int(input("Enter Age:"))
percentage=float(input(("Enter Percentage:")))
s=student(name,age,percentage)
s1=student(name,age,percentage)
s.shoeInfo()
s1.shoeInfo()
print("Name:",s.name)
```

```
Enter Name:Karunya
Enter Age:19
Enter Percentage:88
Name: Karunya
Age: 19
Percentage: 88.0
Name: Karunya
Age: 19
Percentage: 88.0
Name: Karunya
```

```
class MyPython:
  def __init__(self):
    print("First Constructor")

  def __init__(self):
    print("Second Constructor")  #python do not support Constructor Overloading

m=MyPython()
```

```
Second Constructor
```

```
class Python:
  def __init__(self):
    self.a=10
    self.b=20
    self.c=30
    self.d=40

  def func(self):
    del self.d

m=Python()
print(m.__dict__)
m.func()
print(m.__dict__)
```

```
{'a': 10, 'b': 20, 'c': 30, 'd': 40}
{'a': 10, 'b': 20, 'c': 30}
```

```
class MyPython:
  def __init__(self):
    print("Constructor is called")

  def __init__(self):
    print("Destructor is called")

  def func(self):
    print("funs code executed")

m1=MyPython()
m1.func()
```

```
Destructor is called
funs code executed
```

```
#static variable
class MyPython:
  x=20        # stat var
  def __init__(self):
    self.y=10 #instant var

m1=MyPython()
m2=MyPython()
print("m1-->",m1.x,m1.y)
print("m2-->",m2.x,m2.y)
MyPython.x=111
m1.y=222
print("m1-->",m1.x,m1.y)
print("m2-->",m2.x,m2.y)
```

```
m1--> 20 10
m2--> 20 10
m1--> 111 222
m2--> 111 10
```

```
class Test:
  def func1(self):
    a=100
    print(a)

  def func2(self):
    b=222
    a=333
    print(a)
    print(b)

t=Test()
t.func1
t.func2
```

```
Test.func2
def func2()
```

```
<no docstring>
```

```
class student:
  def __init__(self)
```

```
accept 5 paper marks, rollno., name;
and cal total and percentage
if percentage >=85, student can take admission
else cannot take admission
use class, object and constructor
```

```
class Student:
    # Constructor to initialize data
    def __init__(self, roll_no, name, marks):
        self.roll_no = roll_no
        self.name = name
        self.marks = marks
```

```
            self.total = sum(marks)
            self.percentage = (self.total / 500) * 100

        # Method to check and print admission status
        def check_admission(self):
            print("\n--- Result ---")
            print("Name:", self.name)
            print("Roll No:", self.roll_no)
            print("Total Marks:", self.total)
            print("Percentage:", self.percentage, "%")

            if self.percentage >= 85:
                print("Admission Status: CAN take admission")
            else:
                print("Admission Status: CANNOT take admission")

    # --- Taking Inputs ---
    roll = input("Enter Roll No: ")
    name = input("Enter Name: ")

    print("Enter marks for 5 papers:")
    marks_list = []
    for i in range(5):
        m = float(input(f"Paper {i+1}: "))
        marks_list.append(m)

    # Creating Object and calling the method
    s1 = Student(roll, name, marks_list)
    s1.check_admission()
    9
```

```
Enter Roll No: 09
Enter Name: karunya
Enter marks for 5 papers:
Paper 1: 90
Paper 2: 0
Paper 3: 89
Paper 4: 89
Paper 5: 98

--- Result ---
Name: karunya
Roll No: 09
Total Marks: 366.0
Percentage: 73.2 %
Admission Status: CANNOT take admission
```

```
# single level inheritance
class A:
  def showA(self):
    print("showA func")

class B(A):
  def showB(self):
    print("showB func")

obj=B()
obj.showA()
obj.showB()
```

```
showA func
showB func
```

```
# multiple level inheritance
class A:
  def showA(self):
    print("showA func")

class B(A):
  def showB(self):
    print("showB func")

class C(B):
  def showC(self):
    print("showC func")

obj=C()
```

```
obj.showA()
obj.showB()
obj.showC()
```

```
showA func
showB func
showC func
```

```
class A:
  def showA(self):
    print("showA func")

class B:
  def showB(self):
    print("showB func")

class C(A,B):
  def showC(self):
    print("showC func")

obj=C()
obj.showA()
obj.showB()
obj.showC()
```

```
showA func
showB func
showC func
```

```
class parent:
  def bike(self):
    print("slendor+")

class child(parent):
  def bike(self):
    print("HB")
```